



ASSIGNMENT 1

Advanced Classification and Clustering on Stroke Prediction Dataset Using Cluster-Based Label Engineering

Student Name: Diviyah Kumara Rajah

Student ID: MRT254029

Lecturer's Name : Mohd Syahid bin Mohd Anuar

1. Introduction

Machine learning techniques are increasingly applied in healthcare to assist with disease prediction and decision-making. This assignment investigates the use of supervised and unsupervised machine learning techniques on a publicly available Stroke Prediction Dataset obtained from Kaggle.

The dataset contains demographic, health, and lifestyle information of patients and is used to predict stroke occurrence. The dataset was selected because it contains both missing values and an imbalanced target variable, making it suitable for advanced machine learning preprocessing techniques.

The objectives of this assignment are:

1. To build a baseline classification model using preprocessed data.
2. To investigate overfitting and model performance.
3. To apply clustering techniques for feature engineering.
4. To generate new cluster-based labels.
5. To compare the performance of classification models using original and cluster-generated labels.
6. To evaluate whether cluster-based labels can mitigate the issues of imbalanced targets and overfitting.

PART 1: DATA PREPARATION AND BASELINE ANALYSIS

2. Dataset Description

The dataset was obtained from Kaggle using the KaggleHub package.

Dataset Information

Item	Description
Dataset	Healthcare Stroke Prediction Dataset
Records	5,110
Features	12
Target Variable	Stroke
Missing Values	BMI column
Imbalanced Target	Yes

The target variable contains two classes (binary classification)

- Stroke = 0 (No Stroke)

- Stroke = 1 (Stroke)

The dataset is highly imbalanced, with non-stroke cases significantly exceeding stroke cases.

Code Used

```
df.shape
... (5110, 12)

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     5110 non-null   int64
1   gender                 5110 non-null   object
2   age                    5110 non-null   float64
3   hypertension           5110 non-null   int64
4   heart_disease          5110 non-null   int64
5   ever_married           5110 non-null   object
6   work_type              5110 non-null   object
7   Residence_type         5110 non-null   object
8   avg_glucose_level      5110 non-null   float64
9   bmi                    4909 non-null   float64
10  smoking_status          5110 non-null   object
11  stroke                  5110 non-null   int64
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

Check missing values

```
df.isnull().sum()
```

	0
id	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	201
smoking_status	0
stroke	0

dtype: int64

2. Check imbalance

```
df['stroke'].value_counts()
```

	count
stroke	
0	4861
1	249

dtype: int64

```
df['stroke'].value_counts(normalize=True)
```

	proportion
stroke	
0	0.951272
1	0.048728

dtype: float64

The BMI attribute contains 201 missing values.

The target variable is highly imbalanced, with approximately 95% non-stroke cases and only 5% stroke cases.

3. Data Preprocessing

3.1 Encoding Categorical Variables

Machine learning algorithms require numerical input. Therefore, all categorical variables were transformed into numerical format using one-hot encoding.

Code Used

Encode categorical variables into numeric

```
df_encoded = pd.get_dummies(df)
```

3.2 Missing Value Imputation Using KNNImputer

The BMI variable contained missing values.

K-Nearest Neighbour Imputation (KNNImputer) was applied to estimate missing values based on neighbouring observations. KNN finds similar patients and estimates BMI.

Code Used

Handle missing values using KNNImputer

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)

df_impute = pd.DataFrame(
    imputer.fit_transform(df_encoded),
    columns=df_encoded.columns
)

X = df_impute.drop('stroke', axis=1)
y = df_impute['stroke']
```

KNNImputer preserves relationships between variables better than simple mean or median replacement.

3.3 Handling Imbalanced Data Using SMOTE

The original target variable was highly imbalanced.

Synthetic Minority Oversampling Technique (SMOTE) was used to generate synthetic observations for the minority class so that they are balanced.

Code Used

Handle imbalance using SMOTE

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=0)

X, y = smote.fit_resample(X, y)
```

Train-test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=0
)
```

SMOTE helps prevent the classifier from becoming biased toward the majority class.

4. Baseline Classification Model

A Decision Tree Classifier was selected as the baseline model. The tree learns rules automatically.

Code Used

Train Baseline model using Decision Tree classification

```
from sklearn.tree import DecisionTreeClassifier

model_baseline = DecisionTreeClassifier()

model_baseline.fit(X_train, y_train)
```

▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()

Prediction is to let the model guess.

Prediction

```
y_pred = model_baseline.predict(X_test)
```

Evaluation

Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))

print(confusion_matrix(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

```
... Accuracy: 0.9410353102502571
[[1357  93]
 [ 79 1388]]
              precision    recall  f1-score   support

    0.0         0.94        0.94        0.94        1450
    1.0         0.94        0.95        0.94        1467

 accuracy          0.94
 macro avg         0.94
 weighted avg      0.94
```

Results

Metric	Result
Accuracy	94%
Precision	94%

Recall	94%
F1 Score	94%

The confusion matrix indicates strong classification performance for both classes after SMOTE balancing. 94 out of 100 predictions are correct.

5. Overfitting Analysis

Training and testing accuracy were compared.

Code Used

```
OVERFITTING TEST by comparing train_acc with test_acc

train_acc = model_baseline.score(X_train, y_train)
test_acc = model_baseline.score(X_test, y_test)

print("Training Accuracy:", train_acc)
print("Testing Accuracy:", test_acc)

Training Accuracy: 1.0
Testing Accuracy: 0.9410353102502571
```

Results

Metric	Value
Training Accuracy	100%
Testing Accuracy	94%

Analysis

The model achieved perfect performance on training data but slightly lower performance on testing data. This suggests some degree of overfitting, although the model still demonstrates strong generalization capability.

PART 2: FEATURE ENGINEERING VIA CLUSTERING

6. K-Means Clustering

The original target variable was removed and K-Means clustering was applied to discover hidden structures within the dataset. No labels given.

Code Used

PART 2 FEATURE ENGINEERING USING CLUSTERING

I remove the original target

```
X_cluster = X.copy()

from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=0)

cluster_labels = kmeans.fit_predict(X_cluster)

cluster_labels

array([2, 1, 0, ..., 2, 1, 1], dtype=int32)
```

Generated Clusters

Three cluster groups were identified.

The generated cluster labels are:

- Cluster 0 = Young healthy patients
- Cluster 1 = Older moderate risk patients
- Cluster 2 = High risk patients

7. Creating New Target Labels

The cluster labels were used as an alternative target variable. So, the question now is **can we predict which cluster a patient belongs to instead of will the patient get stroke?**

Code Used

PART 3 SECOND CLASSIFICATION MODEL

New target variable

```
y_new = cluster_labels
```

Train-test split

```
X_train2, X_test2, y_train2, y_test2 = train_test_split(  
    X,  
    y_new,  
    test_size=0.3,  
    random_state=0  
)
```

The newly generated labels represent naturally occurring groupings in the data and provide an alternative learning target that is not affected by the original stroke imbalance.

PART 3: EVALUATION AND REPORTING

8. Second Classification Model

A second Decision Tree Classifier was trained using the cluster-generated labels. But now the target becomes Cluster 0, 1 and 2 instead of Stroke or No Stroke

Code Used

Train second model

```
model_cluster = DecisionTreeClassifier()  
model_cluster.fit(X_train2, y_train2)
```

▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()

Prediction

Prediction

```
y_pred2 = model_cluster.predict(X_test2)
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
print("Accuracy:", accuracy_score(y_test2, y_pred2))
```

```
print(confusion_matrix(y_test2, y_pred2))
```

```
print(classification_report(y_test2, y_pred2))
```

```
Accuracy: 1.0
```

```
[[1047  0  0]
```

```
 [  0 1013  0]
```

```
 [  0  0 857]]
```

```
      precision    recall  f1-score   support
```

```
0         1.00      1.00      1.00       1047
```

```
1         1.00      1.00      1.00       1013
```

```
2         1.00      1.00      1.00        857
```

```
accuracy          1.00
```

```
macro avg         1.00      1.00      1.00
```

```
weighted avg      1.00      1.00      1.00
```

Results

Metric	Result
Accuracy	100%
Precision	100%
Recall	100%
F1 Score	100%

9. Comparison of Both Models

Metric	Baseline Model	Cluster-Based Model
Target	Stroke	Cluster Labels
Accuracy	94%	100%
Precision	94%	100%
Recall	94%	100%
F1 Score	94%	100%

Analysis

The baseline model predicts actual stroke occurrence, while the second model predicts cluster-generated labels.

The cluster-based model achieved perfect performance because the labels were generated directly from the same feature space used for training. Therefore, the cluster-based results should not be interpreted as an improvement in stroke prediction performance.

Instead, the results demonstrate the effectiveness of cluster-based label engineering for representing hidden patterns within the dataset.

10. Evaluation of Cluster-Based Labels

The New Labels solved the Imbalanced Target Problem.

The cluster labels created a more balanced target distribution compared to the original stroke variable.

Did the New Labels Reduce Overfitting?

Partially.

The second model achieved perfect performance due to the relationship between the generated labels and the original feature space.

Therefore, while the clustering process produced useful alternative labels, the results should be interpreted carefully because the labels were derived directly from the same dataset used for training.

11. Conclusion

This assignment successfully applied both supervised and unsupervised machine learning techniques to a healthcare stroke prediction dataset.

The baseline classification model achieved approximately 94% accuracy after applying KNN Imputation and SMOTE balancing. K-Means clustering was then used to generate new cluster labels, which served as an alternative target variable for a second classification model.

The cluster-based classification model achieved perfect accuracy when predicting the generated cluster labels. This demonstrates the ability of clustering techniques to identify hidden structures and generate alternative representations of data.

Overall, the study highlights the importance of preprocessing, imbalance handling, feature engineering, and clustering techniques in machine learning workflows.